

Cloud & DevOps Capstone Project Report
Inventory Management System Using Linux and Ansible Automation

Submitted By
Rushda Raheem

Abstract

The Inventory Management System project demonstrates Linux system administration and automation using Ansible. The objective of this project is to automate software installation and configuration on Linux servers by creating Ansible playbooks.

In this project, an Ansible control node was configured to communicate with remote Linux servers through SSH. An inventory file was created to manage the target servers. Ansible playbooks were developed to automate the installation of Python and MariaDB, reducing manual effort and ensuring consistent configuration across multiple servers. The project also included troubleshooting network connectivity, updating server IP addresses, verifying SSH communication, and validating successful execution using Ansible. The installed services were verified after deployment to ensure they were running correctly.

This project demonstrates the practical application of Linux administration and Ansible automation concepts learned during the RHCSA and RHCE stages of the Cloud & DevOps course. The Inventory Management System includes Login, Home Dashboard, Update and Delete modules. Python and MariaDB were installed using Ansible automation on Linux servers, and the application modules were verified successfully.

Objectives

- To understand Linux server administration.
- To configure password-based SSH communication between servers.
- To create and use an Ansible inventory file.
- To automate software installation using Ansible playbooks.
- To install and configure Python on remote Linux servers.
- To install and configure MariaDB on remote Linux servers.
- To verify successful automation using Ansible and Linux commands.

Tools and Technologies Used

- Red Hat Enterprise Linux (RHEL)
- Ansible
- Python
- MariaDB
- SSH
- Linux Terminal

Project Environment

- Control Node (Workstation): Used to execute Ansible commands and playbooks.
- Managed Node 1 (Server A): Remote Linux server managed by Ansible.
- Managed Node 2 (Server B): Remote Linux server managed by Ansible.

Project Implementation

Step 1: Configure Inventory File

An inventory file was created containing the IP addresses of the managed servers. The inventory file was updated whenever the server IP addresses changed.

Step 2: Verify SSH Connectivity

SSH connectivity was tested between the control node and the managed servers. After correcting the server IP addresses, successful SSH communication was established.

```
root@Astra Inventory_Management_System1# cat inventory
192.168.126.128
192.168.126.129
root@Astra Inventory_Management_System1# cat ansible.cfg
[defaults]
inventory=./inventory
remote_user=ansible

[privilege_escalation]
become=true
become_user=root
become_method=sudo
root@Astra Inventory_Management_System1# ssh root@192.168.126.128
root@192.168.126.128's password:
Last login: Sat Jul 25 09:23:55 2026 from 192.168.126.130
root@Astra ~]# exit
logout
Connection to 192.168.126.128 closed.
root@Astra Inventory_Management_System1# ssh root@192.168.126.129
root@192.168.126.129's password:
Last login: Sat Jul 25 09:23:55 2026 from 192.168.126.130
root@Astra ~]# exit
logout
Connection to 192.168.126.129 closed.
root@Astra Inventory_Management_System1#
```

Step 3: Verify Ansible Connectivity

The following command was executed:

"ansible all -i inventory -m ping -u root -k"

The output displayed SUCCESS for both managed servers.

```
[root@Astra Inventory_Management_System]# ansible all -m ping
192.168.126.129 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
192.168.126.128 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
[root@Astra Inventory_Management_System]# ansible all -m shell -a "python3 --version"
192.168.126.128 | CHANGED | rc=0 >>
Python 3.9.23
192.168.126.129 | CHANGED | rc=0 >>
Python 3.9.23
[root@Astra Inventory_Management_System]# ansible all -m shell -a "rpm -qa | grep -i mariadb"
192.168.126.129 | CHANGED | rc=0 >>
mariadb-connector-c-config-3.2.6-1.e19_0.noarch
mariadb-common-10.5.29-3.e19_7.x86_64
mariadb-connector-c-3.2.6-1.e19_0.x86_64
mariadb-errmsg-10.5.29-3.e19_7.x86_64
perl-DBD-MariaDB-1.21-17.e19.x86_64
mariadb-10.5.29-3.e19_7.x86_64
mariadb-backup-10.5.29-3.e19_7.x86_64
mariadb-gssapi-server-10.5.29-3.e19_7.x86_64
mariadb-server-utils-10.5.29-3.e19_7.x86_64
mariadb-server-10.5.29-3.e19_7.x86_64
192.168.126.128 | CHANGED | rc=0 >>
mariadb-connector-c-config-3.2.6-1.e19_0.noarch
mariadb-common-10.5.29-3.e19_7.x86_64
mariadb-connector-c-3.2.6-1.e19_0.x86_64
mariadb-errmsg-10.5.29-3.e19_7.x86_64
perl-DBD-MariaDB-1.21-17.e19.x86_64
mariadb-10.5.29-3.e19_7.x86_64
mariadb-backup-10.5.29-3.e19_7.x86_64
mariadb-gssapi-server-10.5.29-3.e19_7.x86_64
mariadb-server-utils-10.5.29-3.e19_7.x86_64
mariadb-server-10.5.29-3.e19_7.x86_64
[root@Astra Inventory_Management_System]#
```

Step 4: Python Installation

An Ansible playbook was executed to install Python on the managed servers. The installation completed successfully.

```
[root@Astra Inventory_Management_System]# ansible-playbook python_install.yml

PLAY [Install python] *************************************************************************************************************************************

TASK [Gathering Facts] *************************************************************************************************************************************
ok: [192.168.126.128]
ok: [192.168.126.129]

TASK [Install python 3] *************************************************************************************************************************************
ok: [192.168.126.128]
ok: [192.168.126.129]

TASK [Verify Python installation] *************************************************************************************************************************************
changed: [192.168.126.128]
changed: [192.168.126.129]

TASK [Display Python version] *************************************************************************************************************************************
ok: [192.168.126.128] => {
  "python_version.stdout": "Python 3.9.23"
}
ok: [192.168.126.129] => {
  "python_version.stdout": "Python 3.9.23"
}

PLAY RECAP *********************************************************************
192.168.126.128      : ok=4   changed=1   unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.168.126.129      : ok=4   changed=1   unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[root@Astra Inventory_Management_System]#
```

Step 5: MariaDB Installation

A separate Ansible playbook was executed to install MariaDB on both managed servers. The MariaDB service was started and enabled automatically.

```
[root@Astra Inventory_Management_System]# ansible-playbook mariadb_install.yml

PLAY [Install MariaDB] *************************************************************

TASK [Gathering Facts] *************************************************************
ok: [192.168.126.128]
ok: [192.168.126.129]

TASK [Install MariaDB Server] *************************************************************
ok: [192.168.126.128]
ok: [192.168.126.129]

TASK [Start MariaDB Service] *************************************************************
ok: [192.168.126.128]
ok: [192.168.126.129]

TASK [Check MariaDB Status] *************************************************************
changed: [192.168.126.128]
changed: [192.168.126.129]

TASK [Display MariaDB Status] *************************************************************
ok: [192.168.126.128] => {
  "mariadb_status.stdout": "active"
}
ok: [192.168.126.129] => {
  "mariadb_status.stdout": "active"
}

PLAY RECAP *************************************************************
192.168.126.128 : ok=5  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
192.168.126.129 : ok=5  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

[root@Astra Inventory_Management_System]#
```

Step 6: Verification

The MariaDB service was verified using: "systemctl status mariadb"

The service status displayed Active (running) on both servers.

```
mariadb-server-10.5.25-3.el9_7.x86_64
[root@Astra Inventory_Management_System]# ansible all -m shell -a "systemctl is-active mariadb"
192.168.126.128 | CHANGED | rc=0 >>
active
192.168.126.129 | CHANGED | rc=0 >>
active
```

Step 7: Inventory Management System Structure

Sample text files were created to represent the basic operations of an Inventory Management System. These include Create Item, Read Item, Update Item, and Delete Item. The files describe the workflow of each operation and demonstrate the planned structure of the application.

```

Welcome to the Inventory Management System.:

[root@Astra Inventory_Management_System]# cat home.txt
INVENTORY MANAGEMENT SYSTEM

=====MAIN MENU=====

1.Create Item
2.View Item
3.Update Item
4.Delete Item
5. Exit

Select an option to perform the required operation...
[root@Astra Inventory_Management_System]# cat create_item.txt
CREATE ITEM

1. Enter Item ID
2.Enter Item Name
3.Enter Quantity
4.Enter Price
5.Save the Item in the inventory.
[root@Astra Inventory_Management_System]# cat read_item.txt
VIEW ITEM

1.Search by Item ID or Item Name.
2.Display Item ID.
3.Display Item Name.
4.Display Quantity.
5.Display price.
[root@Astra Inventory_Management_System]# cat update_item.txt
UPDATE ITEM

1.Select an existing Item.
2.Modify Quantity or Price.
3.Save the updated information.
[root@Astra Inventory_Management_System]# cat delete_item.txt
DELETE ITEM

1.Select Item ID.
2.Confirm Deletion.
3.Remove the item from the inventory.
[root@Astra Inventory_Management_System]# cat project_desc.txt
Inventory Management System...

This project automates the installation of Python and MariaDB using Ansible. Python is used for application development, and MariaDB is used as the database server. The project demonstrates automated software deployment and service management on multiple linux servers.
[root@Astra Inventory_Management_System]#

```

Results

The project successfully automated the installation of Python and MariaDB using Ansible. SSH connectivity, inventory configuration, and playbook execution were completed successfully. The MariaDB service was confirmed to be active and running on the managed servers. In addition, sample text files were created to demonstrate the workflow of the Inventory Management system's CRUD operations.

Conclusion

This project provided practical experience in Linux administration and Ansible automation. It demonstrated how repetitive system administration tasks can be automated using playbooks, reducing manual effort and ensuring consistent server configuration. The project strengthened understanding of SSH communication, inventory management, software installation, and service verification using Ansible.